

9 have assumed that $\text{P}1$ has its own as ready state and for $\text{P}2$ and $\text{P}3$ we have assumed that the reader or head is at its proper location from where it has to read. ~~9~~.

2) The matrix is

6.4

Time	P1	P3	P5	P7	P8
22	Blocked	Blocked	Running	Blocked suspended due to P2 read	Ready
37	Waiting	Ready/Waiting	Suspended	Blocked due to P5	Running
47	Waiting	Waiting	Running	Suspended	Exited

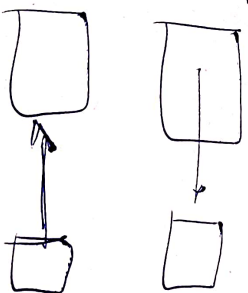
1) a) Suppose for (1) has set S and R to 0

and then ~~but~~ execution goes to $\text{P}1$ since $\text{P}1$ & $\text{P}3$ can't be accessed by $\text{P}1$ and it will be blocked, similarly every time after setting $\text{P}1$ to 0 by for (1) if execution goes to $\text{P}1$ then $\text{P}1$ will be blocked forever since it can't access α and will not be executed -

5) If `foo()` reset the ~~var~~ value of `S` to 1 just after incrementing `S` and then the execution goes to `bar()` then since `R` is `Nil` 0. It can't access `a` and so `foo()` will be executed and similarly if every time after setting the value of `S` if execution goes to `bar()` then it will not be able to execute. So it will be postponed indefinitely.

6) We can see that if a challenge is given by ~~forget~~, then that challenge can't be answered in between.

8) We can set the time limit to answer the challenge and also in between these time this challenge can't be asked by other forgets or wins. This will protect the ~~ans~~ `ans` of attacks will not



END

②. ~~LRU~~

OPTIMAL $\rightarrow$ 1 "perfect" ✓

LRU $\rightarrow$ 2 ✓

second-chance $\rightarrow$ 3 ✓

FIFO $\rightarrow$ 4. ✓

How about anomaly
in LRU, 2nd chance, etc?

(10)

Optimal is always the best as it generates the least number of faults for any input sequence, as it replaces that page which will be used furthest in future.

Least Recently Used is based on temporal locality of memory access. Hence the pages used recently are more likely to be used in near future, as are more likely to be used which were used quite a long time back. Thus, we keep counters / clocks for each LRU page and replace it when required. We can also keep a stack where the most recently used pages are at the top, and the least recently used are at the bottom of the stack. (implemented as doubly-linked list).

Second chance algorithm keeps an access pointer for each page and sets it whenever the page is referenced. Now, while ~~finding~~ finding the page to replace, we check if the pointer value is 0 for any of the pages. If we get such a page, we set it to 1 while searching, if a page has access pointer = 1, we set it to 0. Thus, it is an approximation to LRU algorithm.

FIFO (First-in-First-Out) generally does not perform as good in general cases. ~~also~~ ^{to} ~~it~~ ^{it} also suffers from Belady's Anomaly, i.e., in some request sequences, Page faults increase upon increasing the number of frames in the memory.

LRU ~~second chance~~ and Optimal don't suffer from this anomaly.

In all the cases (FIFO, second chance and LRU) we can create reference/request sequences such that the best FIFO performs the best | second-chance performs the best or etc., but the marking has been done based on average cases.

③ 1-frame $\Rightarrow$ $\begin{matrix} 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20 \end{matrix}$ $\Rightarrow$ all faults

$\Rightarrow$ no page-faults. (any scheme) LRU, FIFO, OPTIMAL

2-frames $\Rightarrow$ $\begin{matrix} 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20 \end{matrix}$

LRU $\rightarrow 20-2 = 18$ page faults

FIFO $\rightarrow$ $\begin{matrix} 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20 \end{matrix}$ $20-2 = 18$ page faults

Optimal $\rightarrow$ $\begin{matrix} 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20 \end{matrix}$ $20-5 = 15$ page faults

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16

3 frames :-
LRU :-
 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16
 = 15 page faults

FIFO :-
 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16
 = 16 page faults

OPTIMAL :-
 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16
 = 11 page faults

4 frames :-
LRU :-
 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16
 = 10 page faults

FIFO :-
 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16
 = 12 page faults

OPTIMAL :-
 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16
 = 8 page faults

5 frames :-
LRU :-
 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16
 = 8 page faults

FIFO :-
 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16
 = 10 page faults

OPTIMAL :-

1	2	3	4	5	6
---	---	---	---	---	---

 $\Rightarrow$ 7 page faults

6 frames
LRU :-

1	2	3	4	5	6
---	---	---	---	---	---

 $\Rightarrow$ 7 page faults

FIFO :-

1	2	3	4	5	6
---	---	---	---	---	---

 $\Rightarrow$ 10 page faults

OPTIMAL :-

1	2	3	4	5	6
---	---	---	---	---	---

 $\Rightarrow$ 7 page faults

7 frames

LRU :-

1	2	3	4	5	6	7
---	---	---	---	---	---	---

 $\Rightarrow$ 7 page faults

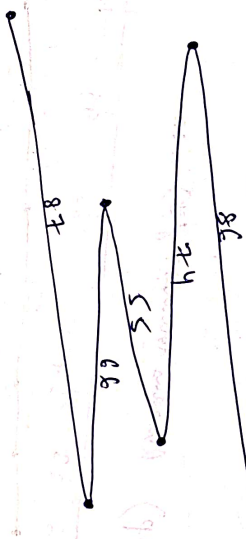
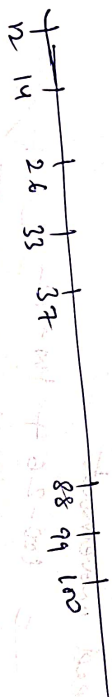
FIFO :- $\Rightarrow$ 7 page faults

OPTIMAL $\Rightarrow$ 7 page faults

(12)

(3)

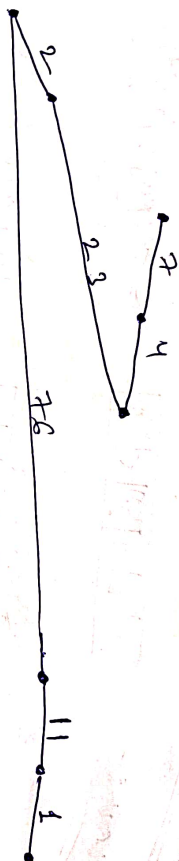
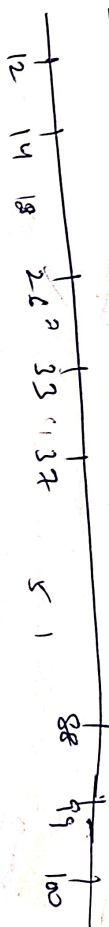
(a) PCFS



Total head movement $\Rightarrow$ 142

$$= (11 + 86 + 86 + 74 + 55 + 66 + 87)$$

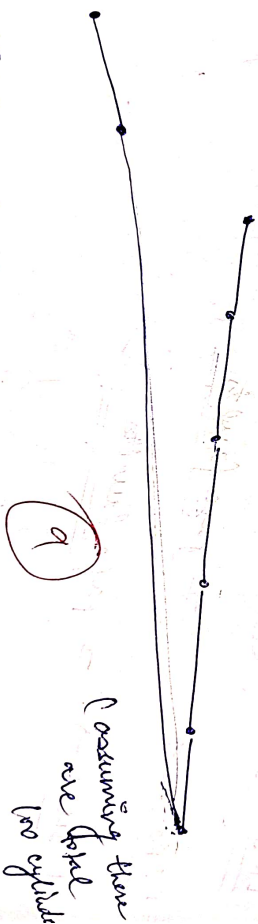
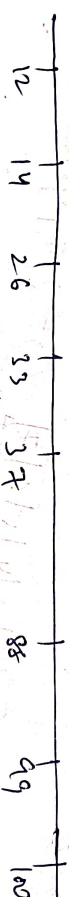
(b) SSTF



Total head movement = $7 + 4 + 25 + 76 + 12$

$$= \boxed{124}$$

(c) SCAN (going up) (means towards right)



(assuming there are 100 cylinders)

Total head movement

$$= 100 - 26 + 100 - 12$$

$$= 200 - 38 = \boxed{162}$$

(d) CSSTF (going up) (means towards right)



$$\text{Total time} = 100 - 26 + 2 = \boxed{76} \times \underline{\underline{88}}$$

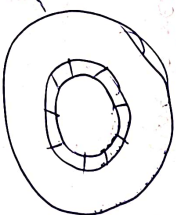
⑥ Rotation speed = 15000 rpm = $\frac{15000}{60} \times \frac{60}{1000} \times 1000$ rpm = 250 rpm.

⇒ 1 rotation in $\frac{1}{250}$ s = 4 ms.

⇒ 0.5 rotation in $\frac{1}{500}$ s = 2 ms

1 sector has 512 bytes

400 sectors per track. $\equiv 204800$ bytes per track.



1000 tracks on a disk.

Average seek time = 4 ms.

File size = 1 M byte = 1048576 bytes.

(File is spread over 6 tracks on disk)

⇒ 400 sectors in 1 rotation.
⇒ (400 × 512) bytes transferred in 1 rotation.

$$\text{Transfer time} = \frac{\text{Transfer size}}{\text{Transfer speed}}$$

$$\Rightarrow \frac{(400 \times 512) \text{ bytes}}{250 \text{ rpm}}$$

(Rotation time)

⑥ Transfer time = $\left(\frac{1048576 \times 4}{400 \times 512} \right) \text{ ms}.$

(6 tracks) = $\frac{2048 \text{ ms}}{20.48 \text{ ms}} = \underline{\underline{100}}$

(b) avg. access time = seek time + rotation delay + Transfer time
= (4 ms + 2 ms) = $\underline{\underline{6 \text{ ms}}}$

c) Rotational delay = Time for $\frac{1}{2}$ rotation = 2ms

d) Time to read 1 sector = Time to read 512 bytes

$$\text{speed} = \frac{400 \times 512}{4} \text{ bytes/ms}$$

$$\Rightarrow \text{Time} = \frac{512 \times 4}{400 \times 512} \text{ ms} = \boxed{0.01 \text{ ms}}$$

e) Time to read 1 track = Time for 1 rotation
 = 4ms $\times$ also to turn to next!

42. 12 direct pointers /

1 indirect pointer /

1 double indirect pointer /

1 triple indirect pointer /

32-bit pointer $\equiv$ 1 block of 8KB.

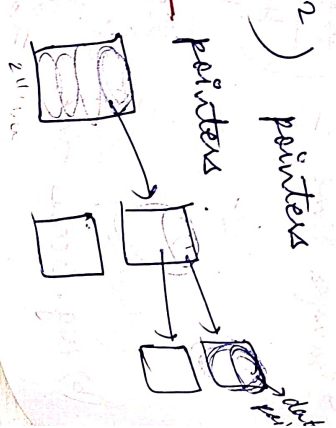
$$\downarrow$$

$$\frac{2^2 \text{ bytes}}{(8 \times 1024)} = \frac{2^{13} \text{ bytes}}{}$$

$\Rightarrow$ 1 block can have $(2^{13} / 2^2)$ pointers

$$= \sqrt{2^{11}} \text{ pointers}$$

1 triple indirect pointer $\Rightarrow$



3 Total file size covered =

$$(\underline{2^{11}} \times \underline{2^{11}} \times \underline{2^{11}}) \text{ blocks} \cdot 20$$

$$= 2^{33} \text{ blocks}$$

$$= 2^{33} \times 8 \text{ KB}$$

$$= 2^{36} \text{ KB} = 2^{16} \text{ GB}$$

$$1 \text{ TB} = 2^{40} \text{ bytes}$$

$$= 2^6 \text{ TB}$$

$$= \boxed{64 \text{ TB}}$$

1 double - indirect pointer covers $\rightarrow 2^{(22)} \text{ blocks}$ 11 and

$$= 2^{25} \text{ KB}$$

$$= 2^5 \text{ GB}$$

$$= 32 \text{ GB}$$

You need to explain the numbers better!

1 indirect pointer covers $\rightarrow 2^{(14)} \text{ KB}$ 14 and

$$= 16 \text{ MB}$$

13 direct pointers cover $\rightarrow 13 \times 8 \text{ KB}$

$$= 104 \text{ KB}$$

Thus, overall file-size covered $\rightarrow \boxed{64 \text{ TB}}$

(10)

⑧ The challenge-response authentication system should be modified such that the target

never solves the challenge.

We should use a Username/login-ID and password-based authentication system, where each user needs to have a login-ID and a corresponding password (never to be disclosed to anyone). Now, even if an attacker knows the login ID, it won't be authenticated unless the attacker somehow gets the password. Now, supposing that the actual user never tells the attacker the password, there is no way the attacker can figure it out; since it is only the target who knows the password other than the user and by no means, the target is going to respond to any request of an attacker in the form of a password. O

The password is something that is known only to the user and the target.

This won't work if the attacker is able to watch the communication between two ends. In that case, the attacker may simply replay the captured messages from a previous authentication session.

(4) (a) Yes the 2 processes can result in both being

locked forever.

eg. $\rightarrow$ let P_0 execute till semWait (S)

$\Rightarrow$ it has taken (S).

$\Rightarrow$ it has executed till semWait (R)

and P_1 has taken (R).

$\Rightarrow$ it has taken (R).

Now, for either of the processes, the next step can't execute as P_0 (1) wants R, which is grabbed by P_1 and P_1 needs (S), which is grabbed by P_0 .
Thus, the 2 processes are grabbed by each other.
remain in deadlock forever.

(b) No I don't think that concurrent execution of these 2 processes can lead to indefinite postponement of one of them. Why?

(5)